

ESTRUCTURAS DE DATOS

Práctica de Laboratorio N° 07: Implementación de colas usando arreglos

I. OBJETIVOS DE LA PRÁCTICA:

- Comprender los conceptos fundamentales de las estructuras de datos tipo cola, su comportamiento FIFO (First In, First Out) y su utilidad en la gestión de procesos y almacenamiento de datos en orden de llegada.
- Implementar la estructura de datos cola utilizando arreglos estáticos, aplicando los principios de la programación orientada a objetos: encapsulamiento, métodos y atributos privados, y la interacción mediante métodos públicos.
- Desarrollar operaciones básicas de una cola como insertar, eliminar, mostrar, contar, buscar y verificar el elemento del frente, reflejando el comportamiento FIFO (First In, First Out) y fortaleciendo la lógica algorítmica y la organización modular del código.

II. DESARROLLO DE LA PRÁCTICA:

1. Una **cola** es una estructura de datos lineal que sigue el principio **FIFO (First In, First Out)**. Al igual que las pilas, se puede implementar con **arreglos o listas enlazadas**. En esta práctica implementaremos una cola utilizando arreglos estáticos. Se creará una clase llamada Cola e implementarán los siguientes métodos:
 - colaVacia() / colaLlena() – Verifica si la cola está vacía o llena.
 - insertarCola(dato) – Inserta un elemento al final de la cola.
 - eliminarCola() – Elimina el elemento ubicado al frente de la cola.
 - verFrente() – Muestra el valor del primer elemento de la cola sin eliminarlo.
 - mostrarCola() – Muestra todos los elementos desde el frente hasta el final.
 - contarElementos() – Devuelve el número total de elementos presentes en la cola.
 - buscarElemento(dato) – Retorna verdadero si el elemento existe en la cola.

El desarrollo de esta práctica permitirá reforzar los conceptos de estructuras de datos lineales, el uso de clases en C++ y el diseño modular del código. Además, se busca que el estudiante entienda claramente cómo funciona el comportamiento FIFO en aplicaciones reales, como sistemas de atención al cliente, colas de impresión o procesamiento de tareas en orden de llegada.

```
1  #include <iostream>
2  #include <string>
3  #define MAX 10 // Tamaño máximo de la cola
4  using namespace std;
5
6  class Cola {
7  private:
8      string elementos[MAX];
9      int frente;
10     int final;
11
12 public:
13     Cola();
14
15     bool colaVacia();
16     bool colaLlena();
17     void insertarCola(string dato);
18     string eliminarCola();
19     string verFrente();
20     void mostrarCola();
21     int contarElementos();
22     bool buscarElemento(string dato);
23 };
24
25 // Constructor: Inicializa la cola vacía
26 Cola::Cola() {
27     frente = -1;
28     final = -1;
29 }
30
```

ESTRUCTURAS DE DATOS

```
31 // Verifica si La cola está vacía
32 bool Cola::colaVacía() {
33     if (frente == -1){
34         return true;
35     } else{
36         return false;
37     }
38 }
39
40 // Verifica si La cola está llena
41 bool Cola::colaLlena() {
42     if (final == MAX - 1){
43         return true;
44     } else{
45         return false;
46     }
47 }
48
49 // Agrega un elemento a la cola
50 void Cola::insertarCola(string dato) {
51     if (!colaLlena()) { //Verifica que hay espacio libre
52         if (colaVacía()){//Se insertará el primer elemento en la cola
53             frente = 0;
54             final = 0;
55         }else{
56             final++;
57         }
58         elementos[final] = dato;
59         cout << dato << " agregado a la cola." << endl;
60     }else{
61         cout << "Desbordamiento - La cola está llena.\n";
62         return;
63     }
64 }
65
66 // Elimina un elemento de la cola
67 string Cola::eliminarCola() {
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85 // Muestra el primer elemento de la cola
86 string Cola::verFrente() {
87
88
89
90
91
92
93
94
95
96 // Muestra todos los elementos de la cola
97 void Cola::mostrarCola() {
98
99
100
101
102
103
104
105
106
107
108
109 // Retorna el número de elementos en la cola
110 int Cola::contarElementos() {
111
112
113
114
115
116 // Busca un elemento en la cola
117 bool Cola::buscarElemento(string dato) {
118
119
120
121
122
123
124
125
126 void menu(){
127     Cola cola1;
128     string dato;
129     int opcion;
130
131     do {
132         cout << "\n--- MENÚ DE COLA ---\n";
133         cout << "1. Insertar elemento en la cola\n";
134         cout << "2. Eliminar elemento de la cola\n";
135         cout << "3. Ver frente\n";
136         cout << "4. Mostrar cola\n";
137         cout << "5. Contar elementos\n";
138         cout << "0. Salir\n";
139         cout << "Seleccione una opción: ";
140         cin >> opcion;
```

ESTRUCTURAS DE DATOS

```
141
142
143     switch (opcion) {
144         case 1:
145             cout << "Ingrese dato a insertar en la cola: ";
146             cin >> dato;
147             cola1.insertarCola(dato);
148             break;
149         case 2:
150             dato = cola1.eliminarCola();
151             if (dato != "")
152                 cout << "Elemento eliminado: " << dato << endl;
153             break;
154         case 3:
155             dato = cola1.verFrente();
156             if (dato != "")
157                 cout << "Elemento en el frente: " << dato << endl;
158             break;
159         case 4:
160             cola1.mostrarCola();
161             break;
162         case 5:
163             cout << "Total de elementos en la cola: " << cola1.contarElementos() << endl;
164             break;
165         case 0:
166             cout << "Programa finalizado.\n";
167             break;
168         default:
169             cout << "Opción inválida.\n";
170     }
171     } while (opcion != 0);
172
173 // Función principal para probar la cola
174 int main() {
175     setlocale(LC_ALL, "");
176     menu();
177     /*Cola cola1;
178
179     cout << "Agregando elementos a, b, c a la cola...\n";
180     cola1.insertarCola("a");
181     cola1.insertarCola("b");
182     cola1.insertarCola("c");
183
184     cola1.mostrarCola();
185
186     cout << "Elemento en el frente: " << cola1.verFrente() << endl;
187
188     cout << "¿Contiene el a?: " << (cola1.buscarElemento("a") ? "Sí" : "No") << endl;
189     cout << "¿Contiene el d?: " << (cola1.buscarElemento("d") ? "Sí" : "No") << endl;
190
191     cout << "Eliminando un elemento de la cola...\n";
192     string dato;
193     dato = cola1.eliminarCola();
194     cout << "elemento eliminado:"<<dato<< endl;
195
196     cola1.mostrarCola();
197
198     cout << "Total de elementos en la cola: " << cola1.contarElementos() << endl;
199     */
200     return 0;
201 }
```

2. Acaba de ocurrir un accidente y se debe evacuar a todos los pasajeros. Para evacuar de forma ordenada creas las siguientes colas:

pasajeros_prioritarios

pasajeros

tripulacion

Estas colas se acaban de llenar con la información de todos los pasajeros, quedando en la primera todos los pasajeros que tienen prioridad al evacuar, luego los demás pasajeros y por último la tripulación. Es tu deber como capitán vaciar estas colas en orden, vaciando primero la cola de pasajeros_prioritarios, luego la de pasajeros y por último la de tripulacion. Cada una de estas colas la deberás vaciar por orden de llegada. Puedes asumir que las colas se llenaron utilizando insertarCola. Realiza un programa que vacíe las colas correctamente.